Chat with Energy Documents (RAG)

Course Name: Gen AI

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no.	Student Name	Enrollment No.
1.	Mahak Shrimal	EN22CS301566
2.	Mahak Taose	EN22CS301567
3.	Mohit Patel	EN22CS301610
4.	Naina Manghani	EN22CS301626
5.	Paavan Patni	EN22CS301671

Group Name:Group 02D6

Project Number:GAI-13

Industry Mentor Name:

University Mentor Name:Prof.Ajaj Khan

Academic Year:2025-2026

TABLE OF CONTENTS

S.NO.	TOPICS	PAGE NO.
	TITLE PAGE	1
	TABLE OF CONTENTS	2-3
	LIST OF FIGURES	4
1.	**Chapter 1:PROBLEM STATEMENT AND OBJECTIVES**	5
	1.1 Problem Statement	5-6
	1.2 Project Objectives	6-7
	1.3 Scope of the Project	7-8
2.	**Chapter 2:PROPOSED SOLUTION**	9
	2.1 Key Features	9-10
	2.2 Overall Architecture / Workflow	10
	2.2.1 System Architecture	10-11
	2.2.2 DFD	11-12
	2.2.3 Use Case Diagram	12
	2.2.4 Class Diagram	13
	2.2.5 Activity Diagram	14
	2.2.6 Flow Chart	15
	2.2.7 Sequence Diagram	16
	2.2.8 Component Diagram	17
	2.3 Tools and Technologies Used	18-19
3.	**CHAPTER 3:RESULTS AND OUTPUT**	20

	3.1 Screenshots / Outputs	20
	3.1.1 APPLICATION STARTUP LOGS	20
	3.1.2 FAST API SWAGGER UI INTERFACE	20-21
	3.1.3 DOCUMENT UPLOAD API EXECUTION	21
	3.1.4 QUERY INPUT VIA API	21-22
	3.1.5 QUERY EXECUTION AND ANSWER OUTPUT	22
	3.1.6 GITHUB REPOSITORY STRUCTURE	22-23
	3.2 REPORTS / DASHBOARDS / LOGS	23
	3.2.1 API LOGS	23
	3.2.2 APPLICATION STARTUP LOGS	23
	3.2.3 DOCUMENT PROCESSING LOGS	23
	3.2.4 EMBEDDING GENERATION LOGS	23
	3.2.5 VECTOR DATABASE LOGS	24
	3.2.6 QUERY PROCESSING LOGS	24
	3.2.7 ANSWER GENERATION LOGS	24
	3.2.8 ERROR HANDLING LOGS	24
	3.3 KEY OUTCOMES	24-25
4.	**CHAPTER 4:CONCLUSION**	26
5.	**CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS**	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Figure 2.2.2	DFD	11-12
Figure 2.2.3	Use Case	12
Figure 2.2.4	Class Diagram	13
Figure 2.2.5	Activity Diagram	14
Figure 2.2.6	Flow Chart	15
Figure 2.2.7	Sequence Diagram	16
Figure 2.2.8	Component Diagram	17
Figure 3.1.1	APPLICATION STARTUP LOGS	20
Figure 3.1.2	FAST API SWAGGER UI INTERFACE	20-21
Figure 3.1.3	DOCUMENT UPLOAD API EXECUTION	21
Figure 3.1.4	QUERY INPUT VIA API	21-22
Figure 3.1.5	QUERY EXECUTION AND ANSWER OUTPUT	22
Figure 3.1.6	GITHUB REPOSITORY STRUCTURE	22-23

CHAPTER 1: PROBLEM STATEMENT & OBJECTIVES

This chapter introduces the problem addressed by the project and defines the goals to be achieved. It explains the limitations of existing systems in handling energy-related documents and highlights the need for an intelligent solution using Generative AI. Additionally, it outlines the key objectives of the project, focusing on building an efficient Retrieval-Augmented Generation (RAG) system for accurate and context-aware document-based question answering.

1.1 PROJECT STATEMENT

In the modern digital era, industries such as energy, power generation, renewable resources, and sustainability produce a vast amount of structured and unstructured data in the form of technical documents, research papers, reports, manuals, and operational guidelines. These documents contain highly valuable domain-specific knowledge, but extracting meaningful insights from them remains a significant challenge.Traditional document search and information retrieval systems are primarily based on keyword matching techniques. While these systems can locate documents containing specific words, they fail to understand the context, intent, and semantics behind user queries. As a result, users often receive irrelevant or incomplete information, even when the correct answer exists within the documents.In the energy sector specifically, documents are often lengthy, complex, and filled with technical terminology. Engineers, researchers, and analysts must spend considerable time manually reading and analyzing these documents to find relevant information. This manual process is not only time-consuming but also inefficient, especially when dealing with large-scale datasets or multiple documents.Another major limitation arises with the use of Large Language Models (LLMs) such as GPT. While these models are highly capable of generating human-like responses, they rely on pre-trained knowledge and may not have access to the latest or domain-specific documents. This leads to a critical issue known as hallucination, where the model generates responses that are incorrect, misleading, or not grounded in actual data.To overcome these limitations, there is a need for a system that can combine the strengths of both information retrieval systems and generative AI models. Such a system should be capable of:

1. Understanding user queries in natural language
2. Retrieving relevant information from uploaded documents
3. Generating accurate, context-aware responses based only on retrieved content

The proposed solution, Chat with Energy Documents using Retrieval-Augmented Generation (RAG), addresses this problem by integrating document retrieval with LLM-based answer generation. The system processes uploaded PDF documents, converts them into vector embeddings, and stores them in a vector database. When a user asks a question, the system retrieves the most relevant document chunks and uses them as context for generating accurate answers.This approach ensures that responses are factually grounded, domain-specific, and reliable, making it highly suitable for applications in the energy sector where accuracy and precision are critical.

Key Problems Identified in the Existing System

1. Lack of Context-Aware Search

Traditional systems rely heavily on keyword-based matching, which does not capture the actual meaning of the query.

A. Keyword-based systems match exact words rather than intent
B. Cannot understand synonyms or semantic meaning
C. Fail to interpret complex or natural language queries

D. Provide multiple irrelevant results instead of a precise answer

2. Manual Effort in Document Analysis

Users are required to manually go through large documents to extract useful information.

A. Energy documents are lengthy and technical
B. Requires domain expertise to interpret content
C. Time-consuming and inefficient process
D. High chances of missing important information

3. Inefficiency in Handling Large Data

As the number of documents increases, traditional systems become less effective.

A. Difficulty in managing large document collections
B. Slow retrieval of relevant information
C. No structured mechanism for organizing knowledge
D. Poor scalability

4. Lack of Intelligent Question-Answering System

Existing systems do not support interactive querying.

A. No conversational interface
B. Users cannot ask questions in natural language
C. No real-time response generation
D. Limited user interaction

1.2 PROJECT OBJECTIVES

The primary objective of the Chat with Energy Documents (RAG) system is to design and implement an intelligent document interaction platform that leverages Generative AI and Retrieval-Augmented Generation (RAG) to provide accurate, context-aware answers from domain-specific documents.Unlike traditional systems, this project focuses on combining semantic retrieval, vector databases, and Large Language Models (LLMs) to ensure that responses are not only relevant but also grounded in actual document content.The objectives are designed to address real-world challenges in document intelligence, improve user experience, and enhance system efficiency.

1. To Develop an Intelligent Document Q&A System

The system aims to allow users to interact with documents in a conversational manner.

A. Enable users to upload energy-related PDF documents
B. Provide an interface to ask questions in natural language
C. Return precise answers instead of full documents
D. Improve accessibility of complex information

2. To Implement Retrieval-Augmented Generation (RAG)

The core objective is to integrate retrieval with generation.

A. Retrieve relevant document chunks based on user query

B. Provide retrieved context to LLM
C. Generate answers grounded in retrieved data
D. Reduce hallucination and improve accuracy

3. To Implement Semantic Search Using Embeddings

The system converts textual data into vector representations.

A. Transform text into embeddings using OpenAI API
B. Capture semantic meaning instead of keywords
C. Enable similarity-based search
D. Improve retrieval accuracy

4. To Design an Efficient Vector Database System

The system uses vector databases for storage and retrieval.

A. Store embeddings in ChromaDB or FAISS
B. Support fast similarity search
C. Retrieve top-k relevant chunks
D. Ensure scalability for large datasets

5. To Enable Accurate Context-Aware Answer Generation

The system ensures answers are based only on relevant data.

A. Use retrieved chunks as context
B. Pass context to LLM for answer generation
C. Avoid generating unsupported information
D. Maintain domain-specific accuracy

1.3 SCOPE OF THE PROJECT

The scope of the project defines the boundaries, functionalities, and limitations of the system. It specifies what the system will do and what it will not cover.

1. Document Upload and Processing

The system allows users to upload documents.

A. Supports PDF file format
B. Validates file input
C. Extracts text using libraries
D. Preprocesses raw data

2. Text Chunking Mechanism

The system divides text into manageable pieces.

A. Splits large text into smaller chunks
B. Maintains context within chunks
C. Uses overlap technique
D. Optimizes chunk size

3. Embedding Generation

Text is converted into vector format.

- **A.** Use OpenAI embedding model
- **B.** Represent text numerically
- **C.** Capture semantic meaning
- **D.** Enable similarity search

4. Vector Database Storage

Embeddings are stored efficiently.

- **A.** Use ChromaDB / FAISS
- **B.** Store document chunks
- **C.** Maintain indexing
- **D.** Enable fast retrieval

5. Query Processing System

The system handles user queries.

- **A.** Accept natural language input
- **B.** Convert query into embedding
- **C.** Perform similarity search
- **D.** Retrieve relevant chunks

Out of Scope

1. Image and Multimedia Processing

- **A.** No image-based document analysis
- **B.** No video/audio processing

2. Multi-Language Support

- **A.** System supports primarily English
- **B.** No translation features

3. Advanced Model Training

- **A.** No custom LLM training
- **B.** Uses pre-trained APIs

4. Enterprise-Level Deployment

- **A.** No distributed microservices
- **B.** No advanced cloud orchestration

5. Real-Time Data Integration

- **A.** No live data streaming

CHAPTER 2: PROPOSED SOLUTION

This chapter describes the proposed solution for the identified problem by presenting the design and implementation of the RAG-based system. It explains the system architecture, workflow, components, and technologies used to process documents and generate context-aware answers. The chapter also includes diagrams and detailed descriptions of how different modules interact to ensure efficient document processing and intelligent query handling.

2.1 KEY FEATURES

The proposed system, Chat with Energy Documents (RAG), provides an intelligent and automated solution for extracting insights from domain-specific documents using Generative AI. The system integrates document processing, semantic search, and Large Language Models to deliver accurate and context-aware responses.

1. Intelligent Document Upload System

The system enables users to upload energy-related documents in PDF format for processing.

- **A.** Supports uploading of single or multiple PDF files
- **B.** Validates file type and ensures correct format
- **C.** Handles large documents efficiently
- **D.** Stores document metadata for tracking

2. Automated Text Extraction and Preprocessing

Once documents are uploaded, the system extracts textual content.

- **A.** Uses PDF processing libraries (pdfplumber / PyPDFLoader)
- **B.** Removes unwanted characters and formatting
- **C.** Cleans and structures raw text
- **D.** Prepares text for further processing

3. Advanced Text Chunking Mechanism

The system divides large documents into smaller chunks for efficient processing.

- **A.** Splits text into manageable segments
- **B.** Maintains context within each chunk
- **C.** Uses overlapping technique to preserve meaning
- **D.** Optimizes chunk size for better retrieval

4. Semantic Embedding Generation

The system converts text into numerical vector representations.

- **A.** Uses OpenAI embedding models
- **B.** Captures semantic meaning of text
- **C.** Converts both documents and queries into vectors
- **D.** Enables similarity-based search

5. High-Performance Vector Database

The system stores embeddings in a vector database for fast retrieval.

- **A.** Uses ChromaDB / FAISS
- **B.** Stores embeddings efficiently
- **C.** Supports similarity search (cosine similarity)
- **D.** Retrieves top-k relevant chunks

6. Intelligent Query Processing System

The system processes user queries using semantic understanding.

- **A.** Accepts natural language queries
- **B.** Converts query into embedding
- **C.** Searches vector database
- **D.** Retrieves most relevant document chunks

2.2 OVERALL ARCHITECTURE / WORKFLOW

2.2.1 SYSTEM ARCHITECTURE

The system follows a layered architecture.

1. User Layer

- **A.** Uploads documents
- **B.** Asks queries
- **C.** Receives responses

2. Application Layer (FastAPI)

- **A.** Handles API requests
- **B.** Manages workflow
- **C.** Coordinates modules

3. Processing Layer

- **A.** Extracts text
- **B.** Cleans data
- **C.** Prepares input

4. Embedding Layer

- **A.** Converts text into vectors
- **B.** Uses OpenAI embeddings

5. Storage Layer

- **A.** Stores embeddings
- **B.** Uses ChromaDB

6. Retrieval + LLM Layer

A. Retrieves relevant chunks
B. Generates answers

2.2.2 DATA FLOW DIAGRAM(DFD)

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It illustrates the flow of information between different components such as processes, data stores, and external entities.DFD helps in understanding how input data is transformed into output through a sequence of operations within the system.

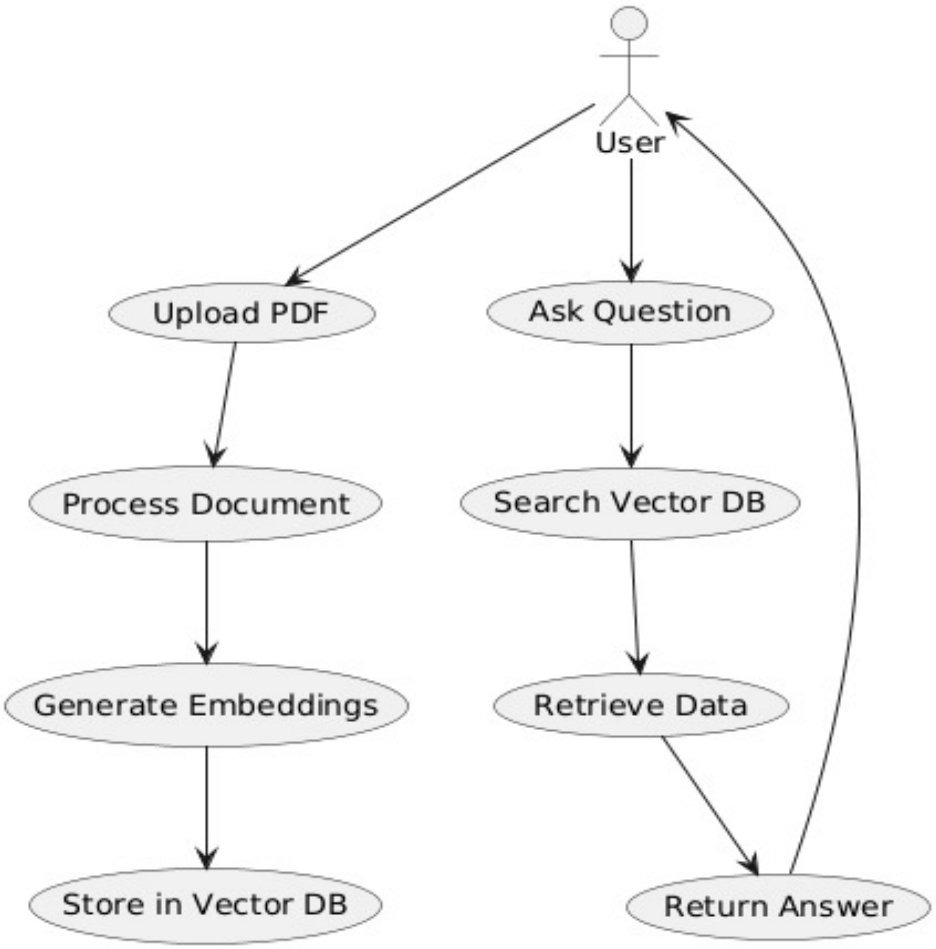

Figure 2.2.2

Working:

The diagram represents the working of the RAG-based Chat with Energy Documents system, showing how the user interacts with the system in two main flows: document upload and query processing.Initially, the process begins when the user uploads a PDF document. The system then processes the document, where the text is extracted and cleaned. After processing, the system generates embeddings, which means converting the text into numerical vector representations that capture the semantic meaning of the content. These embeddings are then stored in the vector database, allowing efficient and similarity-based retrieval later.In the second flow, when the user asks a question, the system takes the query and performs a search in the vector database by converting the

query into an embedding and matching it with stored document embeddings. The system then retrieves the most relevant data (chunks) from the database. Finally, using this retrieved context, the system generates and returns an answer to the user. This entire process ensures that the response is accurate, context-aware, and strictly based on the uploaded document content.

2.2.3 USE CASE DIAGRAM

A Use Case is a description of how a user (actor) interacts with a system to achieve a specific goal. It represents a particular functionality or action that the system performs in response to user input.

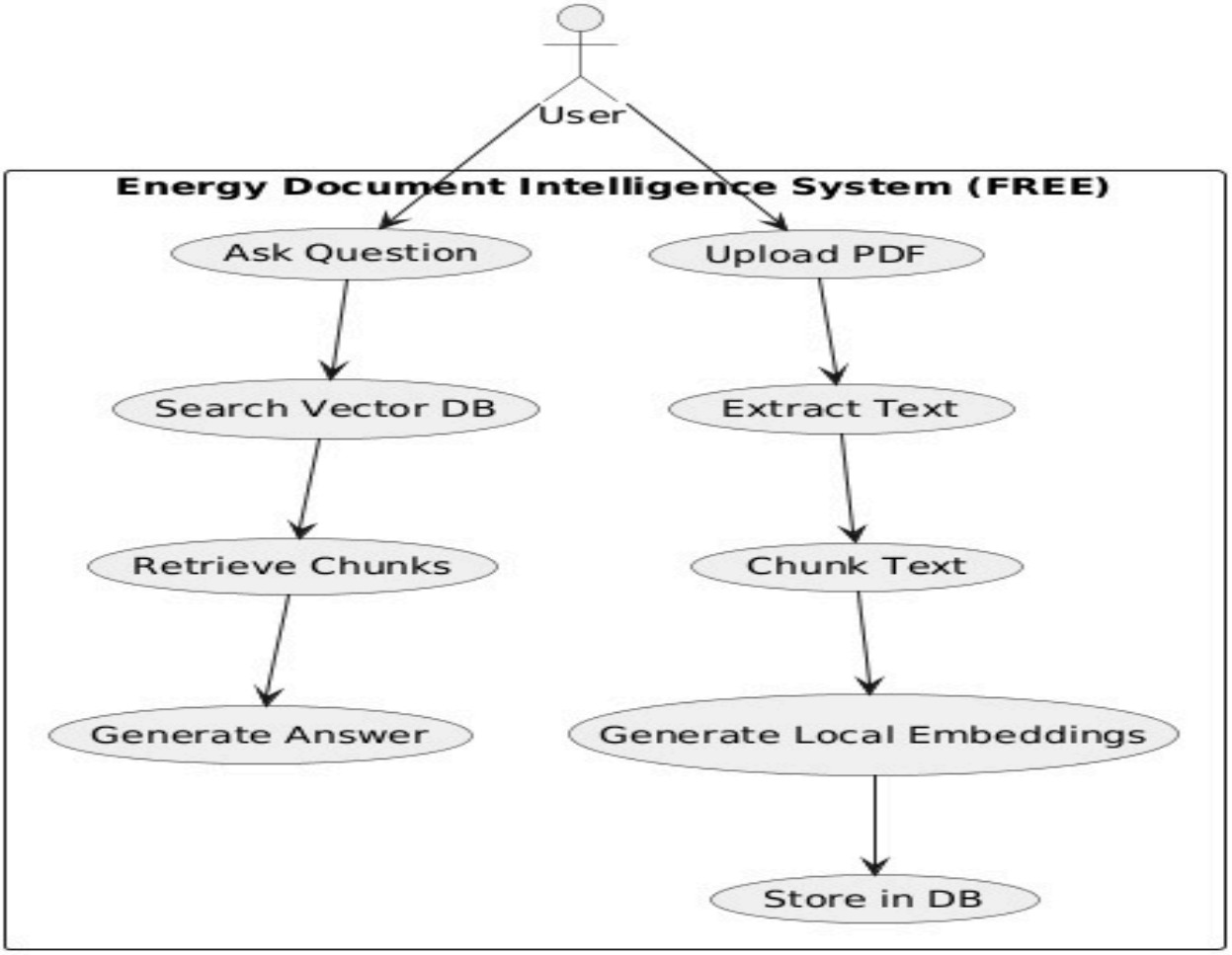

Figure 2.2.3

Working:

The Use Case Diagram represents how the user interacts with the Energy Document Intelligence System (RAG) to perform two main operations: uploading documents and asking questions. The user acts as the primary actor who initiates both processes. When the user uploads a PDF, the system processes the document by extracting text, dividing it into smaller chunks, generating embeddings, and storing them in the vector database. This prepares the system for efficient retrieval. On the other hand, when the user asks a question, the system searches the vector database, retrieves the most relevant chunks, and generates an answer using a language model. The diagram clearly shows how the system transforms user inputs into meaningful outputs through a sequence of automated steps, ensuring accurate and context-aware responses.

2.2.4 CLASS DIAGRAM

A Class Diagram is a type of UML (Unified Modeling Language) diagram that represents the structure of a system by showing its classes, attributes, methods, and relationships between them.It helps in understanding how different components of a system are organized and how they interact with each other.

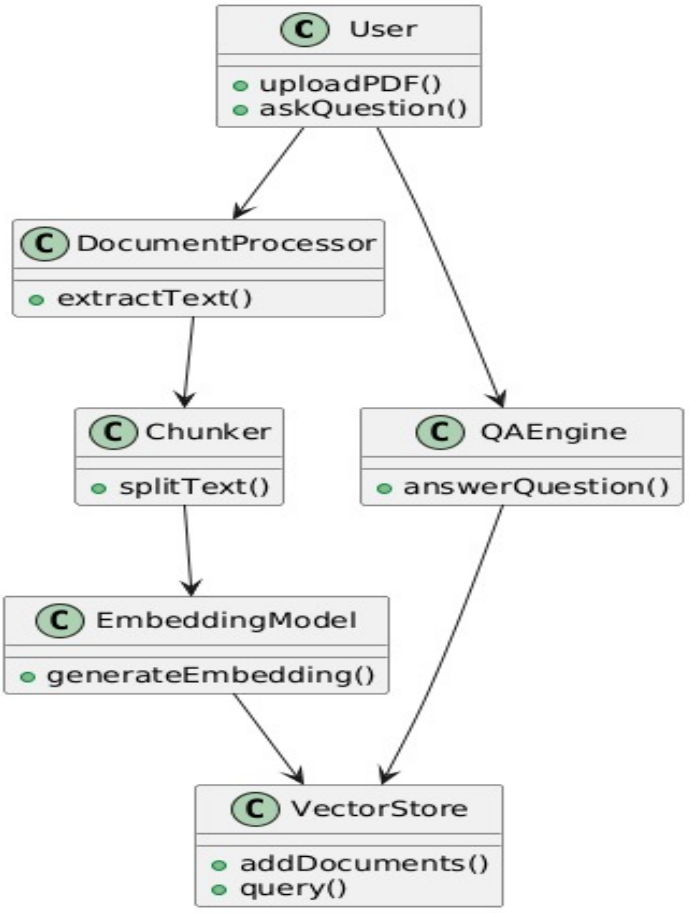

Figure 2.2.4

Working:

The class diagram represents the internal structure and interaction of components in the RAG-based Energy Document Intelligence System. The process begins with the User class, which interacts with the system by uploading PDFs and asking questions. When a PDF is uploaded, the request is handled by the DocumentProcessor class, which extracts text from the document. This extracted text is then passed to the Chunker class, where it is divided into smaller chunks for efficient processing. These chunks are further processed by the EmbeddingModel, which generates vector embeddings representing the semantic meaning of the text. The embeddings are then stored in the VectorStore, which acts as a database for efficient retrieval. When the user asks a question, it is handled by the QAEngine, which queries the VectorStore to retrieve relevant data and generates a context-aware answer. This structured interaction between classes ensures smooth data flow, modularity, and efficient implementation of the RAG pipeline.

2.2.5 ACTIVITY DIAGRAM

An Activity Diagram is a type of UML diagram that represents the flow of activities or steps in a system. It shows how a process starts, what actions are performed, decision points (if any), and how the process ends.

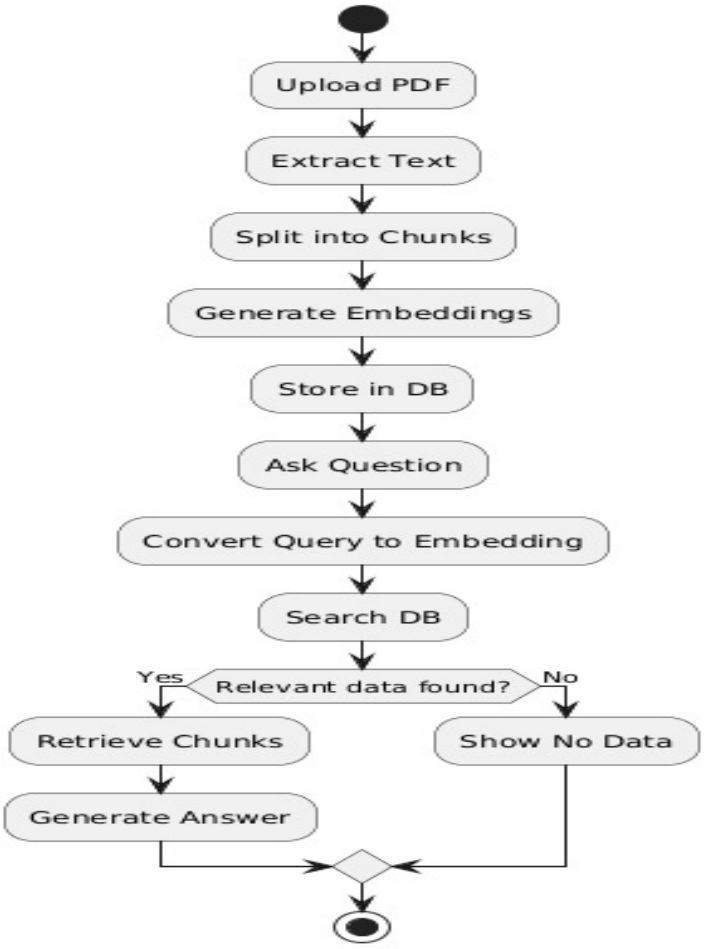

Figure 2.2.5

Working:

The activity diagram represents the complete workflow of the RAG-based Energy Document Intelligence System, starting from document upload to answer generation. The process begins when the user uploads a PDF document, after which the system extracts text from the document and splits it into smaller chunks. These chunks are then converted into embeddings, which are stored in the vector database for efficient retrieval.After storing the data, the user asks a question. The system converts the query into an embedding and searches the vector database to find relevant information. At this stage, a decision point occurs: if relevant data is found, the system retrieves the corresponding chunks and generates an answer using the language model. If no relevant data is found, the system displays a "No Data" message. Finally, the process ends after either generating the answer or showing that no data is available.This diagram clearly shows both the sequential flow and the decision-making process involved in the system.

2.2.6 FLOW CHART

A Flowchart is a graphical representation of a process that shows the sequence of steps using symbols such as rectangles (process), diamonds (decision), and arrows (flow). It helps in understanding the logical flow of a system from start to end.

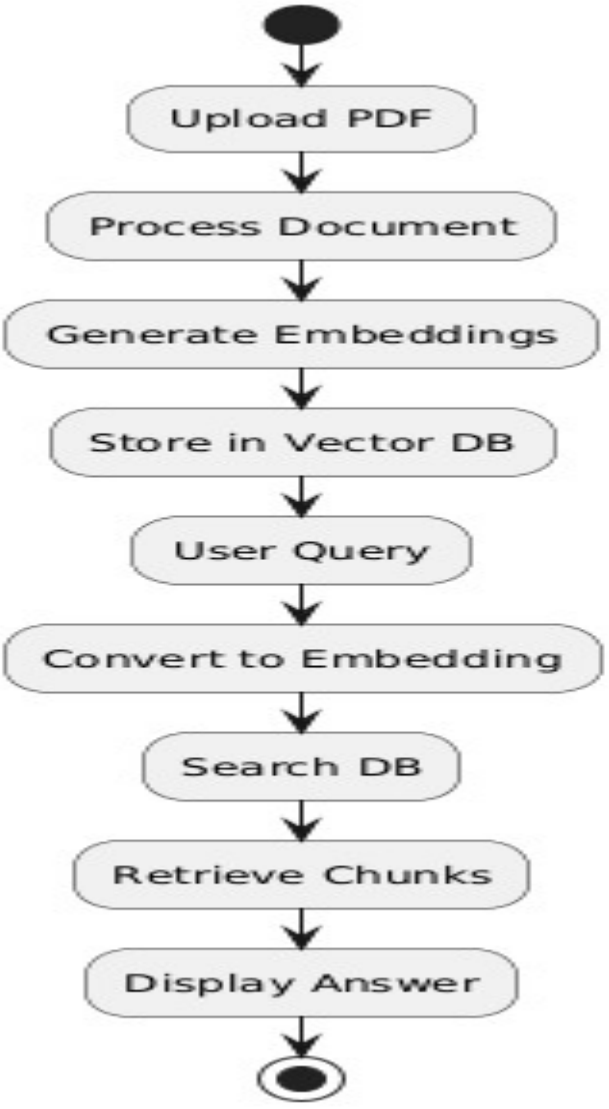

Figure 2.2.6

Working:

The flowchart represents the step-by-step workflow of the Energy Document Intelligence System (RAG) in a simple and linear manner. The process begins when the user uploads a PDF document. The system then processes the document by extracting and preparing the text. After processing, embeddings are generated to convert the text into vector format, which is then stored in the vector database. Once the data is stored, the user submits a query. The system converts this query into an embedding and searches the vector database to find relevant information. It then retrieves the most relevant chunks and uses them to generate a response. Finally, the system displays the answer to the user, and the process ends. This flowchart clearly shows a smooth and sequential flow without decision branches, making it easy to understand the working of the RAG pipeline.

2.2.7 SEQUENCE DIAGRAM

A Sequence Diagram is a UML diagram that shows how different components of a system interact with each other in a specific order over time. It represents the sequence of messages exchanged between system elements to complete a process.

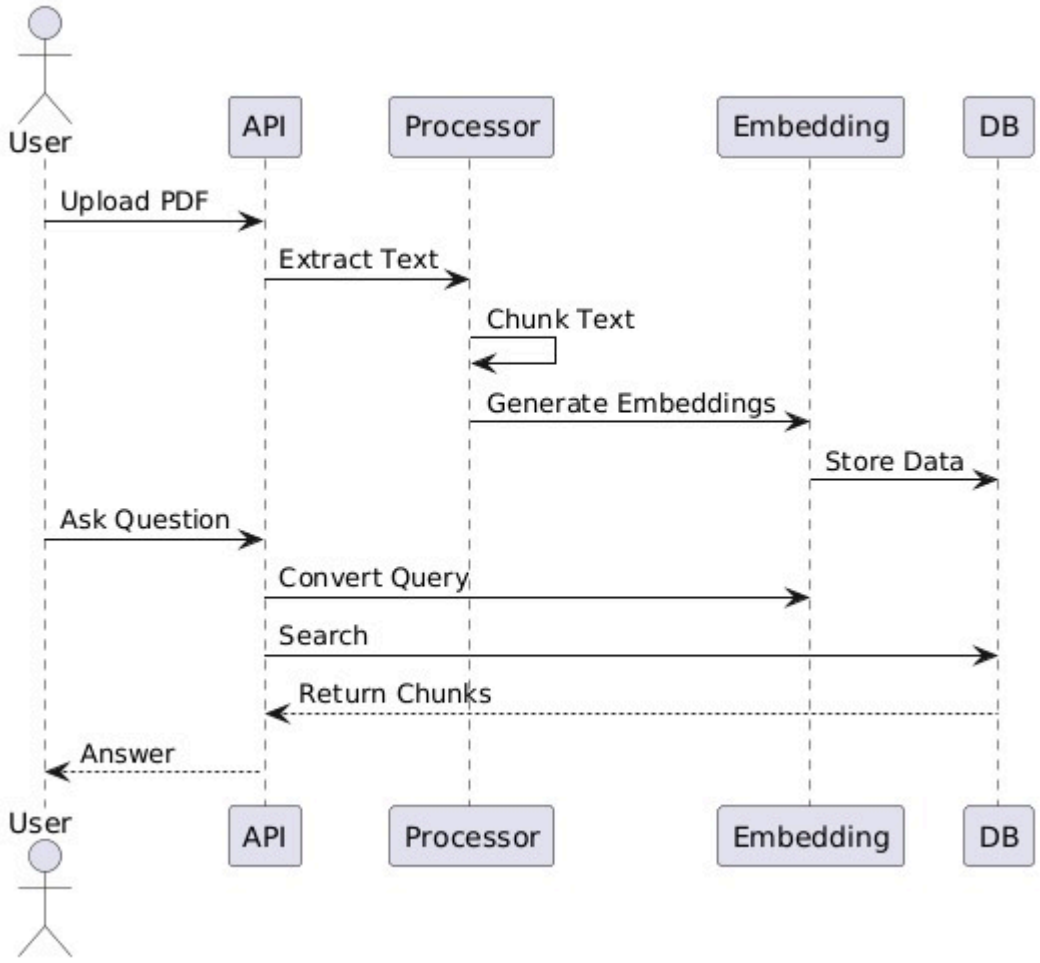

Figure 2.2.7

Working:

The sequence diagram illustrates how different components of the Energy Document Intelligence System (RAG) interact with each other over time to complete both document processing and query answering tasks. The process begins when the user uploads a PDF, which is sent to the API. The API forwards the request to the Processor, which extracts text from the document and performs chunking. The processed text is then sent to the Embedding module, where embeddings are generated and passed to the database (DB) for storage.In the second phase, when the user asks a question, the request is again sent to the API. The API converts the query into an embedding and sends it to the Embedding module. The system then performs a search in the database to find similar embeddings. The database returns the most relevant chunks back to the API, which then generates the final answer and sends it to the user. This diagram clearly shows the sequential communication between components and how data flows step-by-step through the system.

2.2.8 COMPONENT DIAGRAM

A Component Diagram is a type of UML diagram that shows the high-level structure of a system by representing its components (modules) and how they interact with each other.It focuses on the physical and logical parts of the system and their relationships.

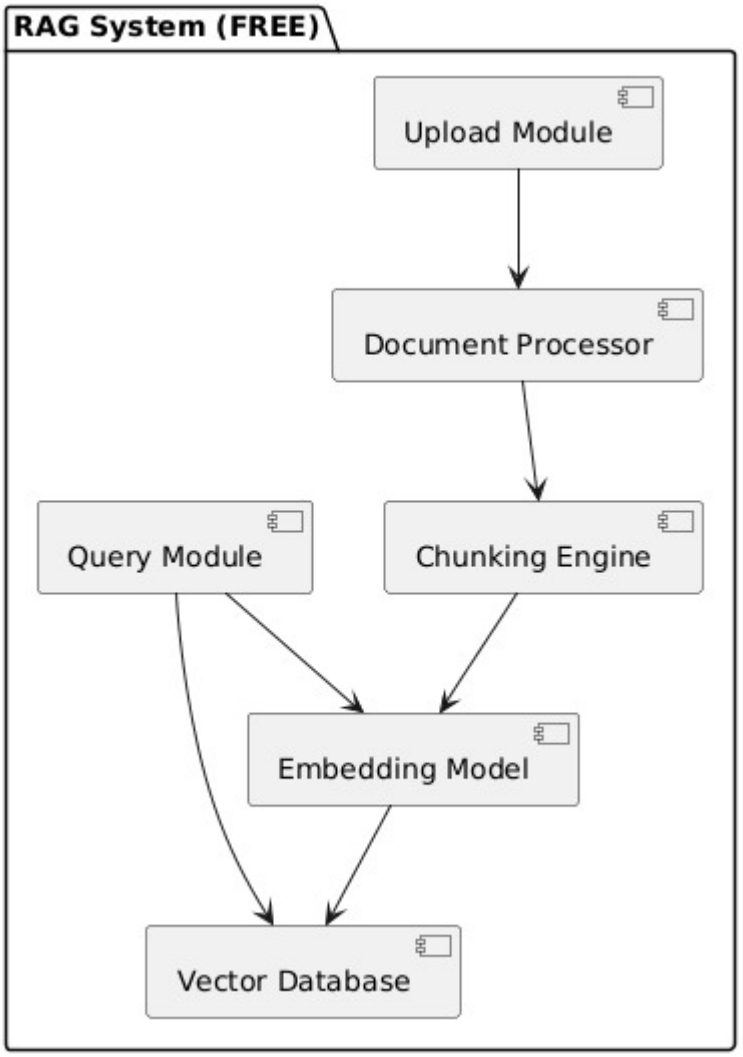

Figure 2.2.8

Working:

The component diagram represents the high-level architecture of the RAG System (Energy Document Intelligence System), showing how different modules interact to process documents and generate answers. The process begins with the Upload Module, where the user uploads a PDF document. This document is then passed to the Document Processor, which extracts and prepares the text. The processed text is forwarded to the Chunking Engine, where it is divided into smaller chunks for efficient handling.These chunks are then sent to the Embedding Model, which converts them into vector representations. The generated embeddings are stored in the Vector Database, enabling fast and semantic retrieval. On the query side, when a user asks a question, it is handled by the Query Module, which also interacts with the Embedding Model to convert the query into vector form. The Embedding Model then communicates with the Vector Database to retrieve relevant information. This modular interaction ensures that both document processing and query handling are efficiently managed within the system.

2.3 TOOLS AND TECHNOLOGIES USED

The development of the Energy Document Intelligence System (RAG) involves the integration of multiple technologies from backend development, artificial intelligence, data processing, and database management. Each tool plays a crucial role in ensuring the system functions efficiently, accurately, and at scale.

1. Backend Technologies

A. **Python-**Implements backend logic.Handles document processing.Integrates APIs and databases
B. **FastAPI-**Provides API endpoints:
/upload → Upload documents
/query → Ask questions
/health → Check system status

2. Artificial Intelligence & NLP Technologies

A. **OpenAI API-**Generate embeddings for text.Generate answers using LLM.Provide context-aware responses
B. **Embedding Models-**Convert document chunks into vectors.Convert user queries into vectors.Enable semantic search

3. Vector Database Technologies

A. **ChromaDB-**Store document embeddings.Retrieve relevant chunks
B. **FAISS-**Perform similarity search

4. Document Processing Technologies

A. **pdfplumber-**Extract text from uploaded documents
B. **Text Splitter (Chunking)-**Split large text into chunks.Improve embedding quality

5. Database & Storage

A. **Vector Storage-**Store embeddings.Enable semantic search

6. API & Communication

A. **REST APIs-**Upload documents.Query system.Return responses

7. Frontend technologies

A. **Streamlit-**Provides file upload interface.Allows users to enter queries.Displays answers generated by the system.Connects directly with backend APIs
B. **HTML (HyperText Markup Language)-**Creates structure of web pages
C. **CSS (Cascading Style Sheets)-**Styles the UI.Improves visual appearance
D. **JavaScript-**Adds interactivity.Handles user actions (click, input)

CHAPTER 3:RESULTS AND OUTPUTS

This chapter presents the results obtained from the implementation of the proposed RAG system. It includes system outputs, API responses, screenshots, and logs that demonstrate the working of document processing, embedding generation, and query handling. The chapter highlights how the system successfully generates accurate and context-aware answers based on the uploaded documents.

3.1 SCREENSHOTS / OUTPUTS

3.1.1 APPLICATION STARTUP LOGS

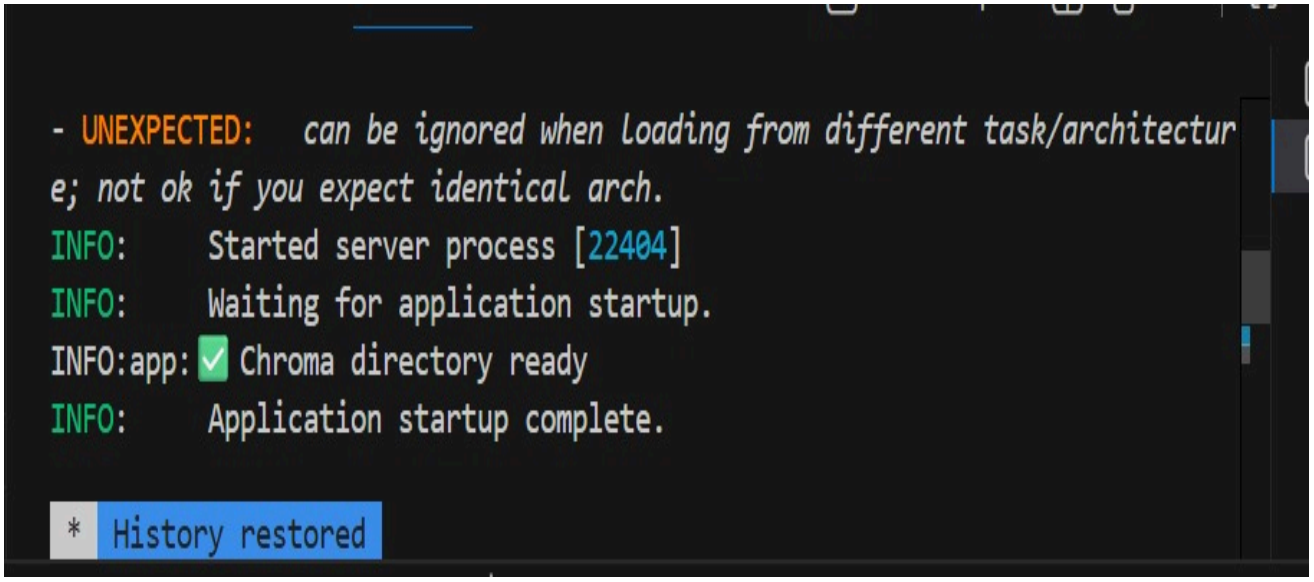

Figure 3.1.1

This screenshot displays the backend server startup logs. It shows that the FastAPI server has successfully started, and the Chroma vector database directory is ready. The message "Application startup complete" confirms that the system is running without errors and is ready to handle requests.

3.1.2 FAST API SWAGGER UI INTERFACE

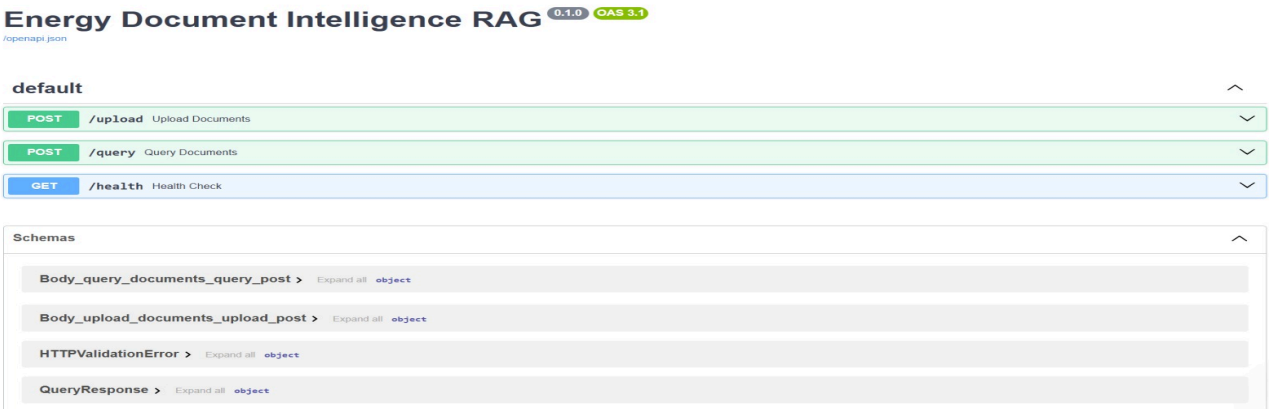

Figure 3.1.2

This screenshot shows the Swagger UI interface provided by FastAPI. It lists all available API endpoints such as /upload, /query, and /health. This interface acts as the frontend for testing the system, allowing users to interact with APIs directly without needing a separate UI.

3.1.3 DOCUMENT UPLOAD API EXECUTION

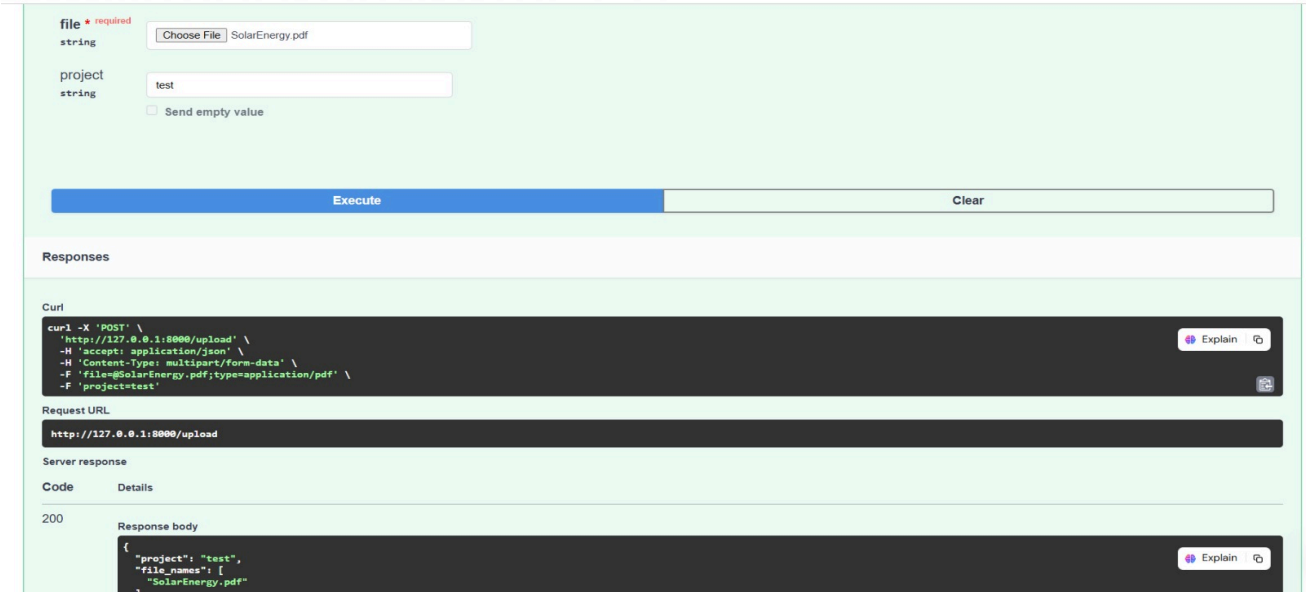

Figure 3.1.3

This screenshot demonstrates the execution of the /upload API. The user selects a PDF file (SolarEnergy.pdf) and provides a project name. Upon execution, the system processes the document, generates embeddings, and stores them in the vector database. The successful response confirms that the document has been uploaded and processed correctly.

3.1.4 QUERY INPUT VIA API

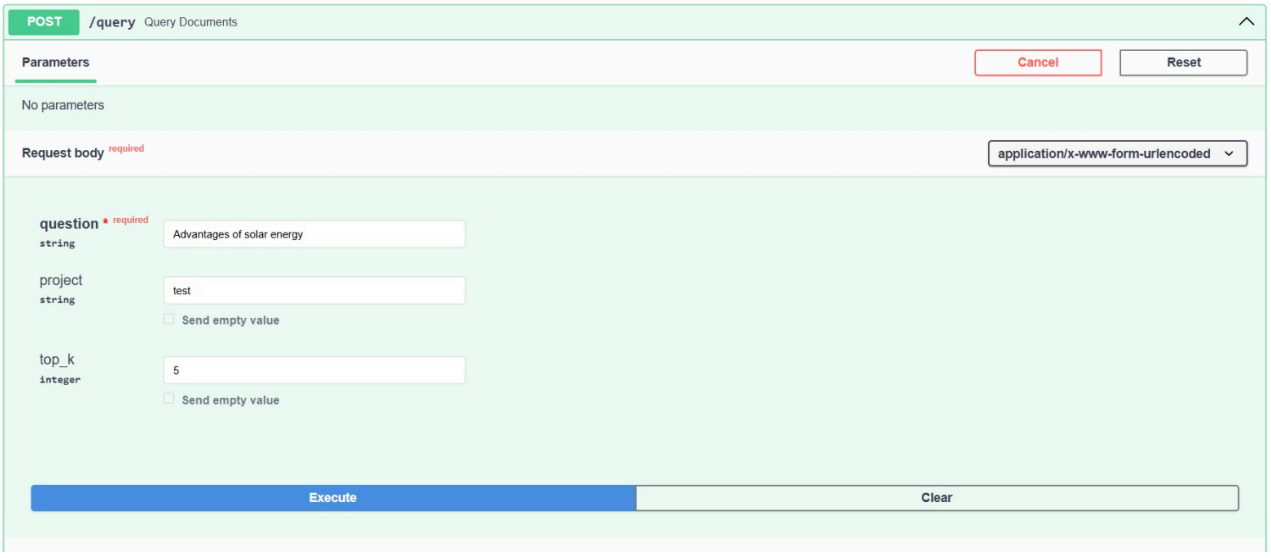

Figure 3.1.4

This screenshot shows the /query API input section where the user enters a question such as "Advantages of solar energy". Additional parameters like project name and top_k (number of results) are provided. This step initiates the retrieval process from the vector database.

3.1.5 QUERY EXECUTION AND ANSWER OUTPUT

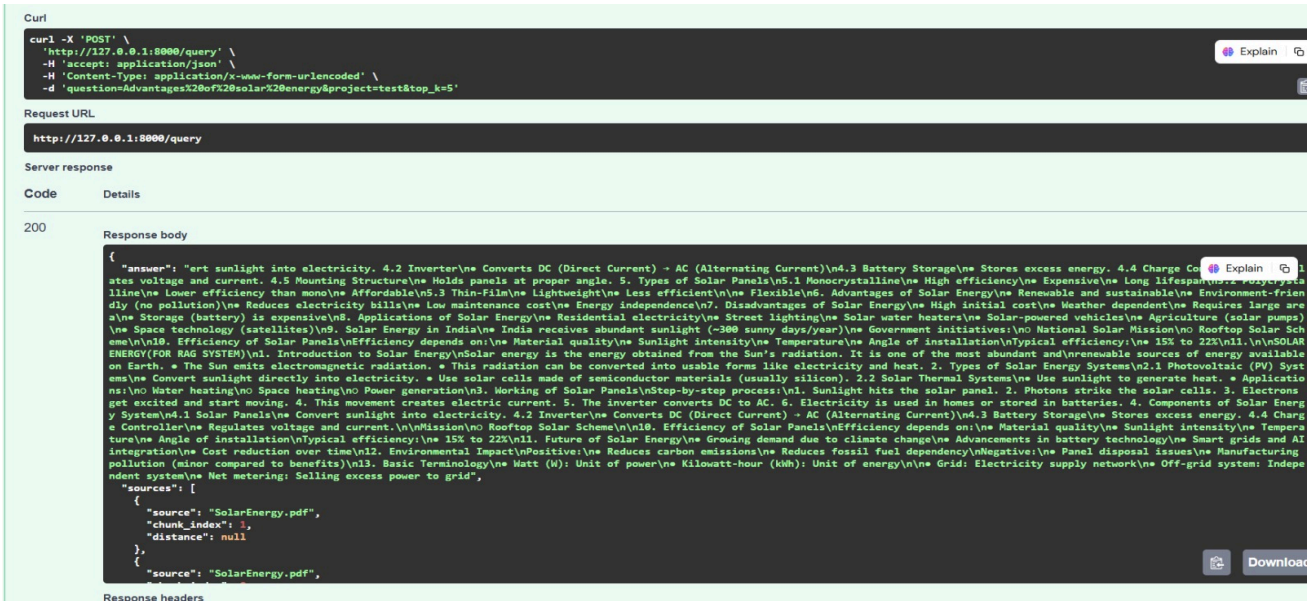

Figure 3.1.5

This screenshot displays the final response generated by the system after executing the /query API. The system retrieves relevant chunks and generates a context-aware answer using the LLM. The output also includes source details, ensuring accuracy and reliability.

3.1.6 GITHUB REPOSITORY STRUCTURE

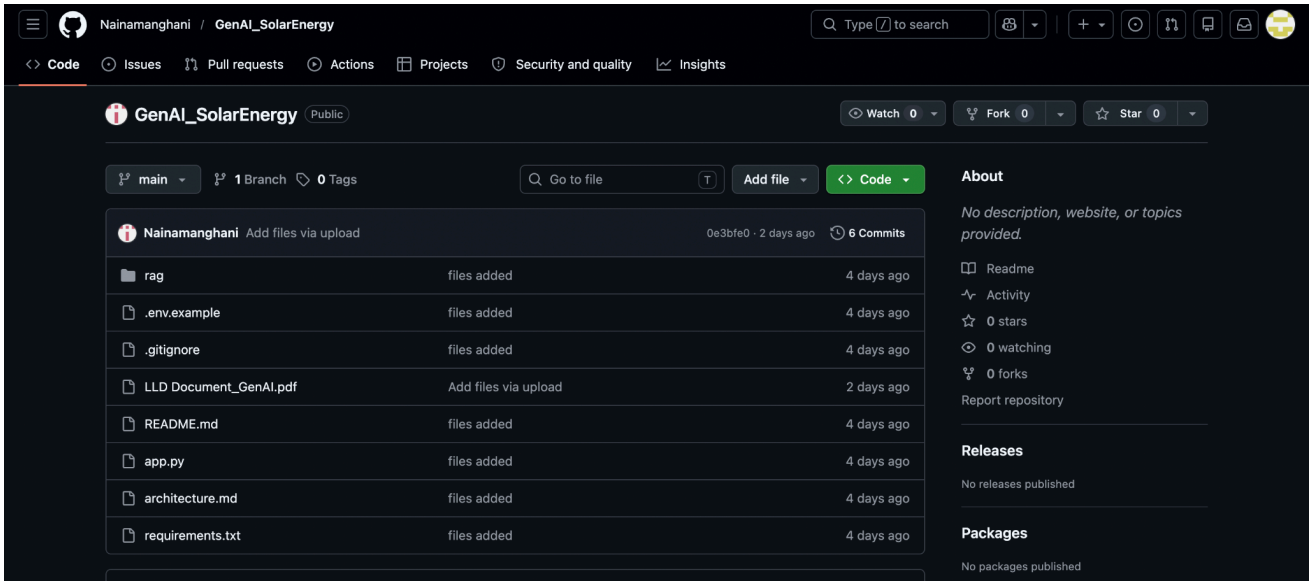

Figure 3.1.6

This screenshot shows the GitHub repository of the project named GenAI_SolarEnergy, which contains all the source code and project files. It includes important files such as app.py (main backend), requirements.txt (dependencies), and folders like rag for modular implementation. The repository ensures proper version control, collaboration, and organized project management.

3.2 REPORTS / DASHBOARDS / LOGS

The Energy Document Intelligence System (RAG) generates various logs and reports during its execution, which help in monitoring system performance, tracking operations, and debugging errors. These logs provide insights into different stages of the pipeline, including document processing, embedding generation, vector storage, and query handling. Proper logging ensures transparency, reliability, and efficient maintenance of the system.

3.2.1 API Logs

API logs track all incoming and outgoing requests handled by the FastAPI backend.Records all API calls such as /upload, /query, and /healthTracks request time and response status.Logs success and failure messages.Helps in debugging API-related issues

Example:

POST /upload → 200 OK

3.2.2 Application Startup Logs

These logs indicate whether the system has started successfully.Shows server initialization.Confirms database readiness.Displays system status

3.2.3 Document Processing Logs

These logs track document-related operations.Confirms file upload.Shows text extraction status.Displays number of chunks created.Tracks preprocessing steps

Example:

File uploaded successfully

3.2.4 Embedding Generation Logs

These logs track embedding creation.Shows embedding generation process.Confirms vector creation.Tracks time taken for embedding

Example:

Embeddings generated for 120 chunks

3.2.5 Vector Database Logs

These logs track storage and retrieval operations.Confirms data stored in vector DB.Tracks indexing process.Logs similarity search operations

Example:

Data stored successfully

3.2.6 Query Processing Logs

These logs track how queries are handled.Records user queries.Tracks query embedding generation.Logs retrieval process

Example:

Query received: "Advantages of solar energy"

3.2.7 Answer Generation Logs

These logs track response generation.Shows LLM response generation.Tracks context usage.Confirms answer delivery

Example:

Answer generated successfully

3.2.8 Error Handling Logs

These logs capture system errors and exceptions.Handles invalid file uploads.Detects missing inputs.Logs API failures.Displays error messages

Example:

Invalid file format

3.3 KEY OUTCOMES

1. Successful Implementation of RAG Pipeline

Complete workflow implemented:

A. Document upload
B. Text extraction
C. Chunking
D. Embedding generation
E. Vector storage
F. Query processing
G. Answer generation

2. Accurate and Context-Based Responses

Answers are generated using retrieved document chunks.Reduces hallucination of LLM.Ensures domain-specific accuracy

3. Efficient Document Processing

Automatic text extraction from PDFs.Effective chunking strategy.Handles large documents

4. Fast and Efficient Retrieval System

Uses vector database (ChromaDB/FAISS).Performs similarity search.Retrieves top-k relevant results

5. Scalable and Modular Architecture

Independent components.Easy to extend system.Supports multiple documents

6. Improved User Experience

Simple API-based interaction.Easy document upload.Quick query response

CHAPTER 4:CONCLUSION

The Energy Document Intelligence System using Retrieval-Augmented Generation (RAG) has been successfully designed and implemented to address the growing need for intelligent document analysis in the energy domain. With the increasing volume of technical documents such as reports, manuals, and research papers, extracting relevant information efficiently has become a significant challenge. This project provides a robust solution by enabling users to interact with such documents through natural language queries, thereby simplifying the process of information retrieval.The system integrates modern technologies including FastAPI for backend development, vector databases such as ChromaDB for storage, and OpenAI-based Large Language Models (LLMs) for generating responses. The implementation follows a structured pipeline where documents are uploaded, processed, and converted into embeddings. These embeddings capture the semantic meaning of the text and are stored in a vector database for efficient retrieval. When a user submits a query, the system converts the query into an embedding, performs a similarity search, retrieves relevant document chunks, and generates an answer using the LLM. This ensures that the responses are context-aware and grounded in the uploaded data.One of the key highlights of this project is the successful implementation of the Retrieval-Augmented Generation (RAG) architecture, which effectively combines the strengths of information retrieval systems and generative AI models. Traditional AI systems often suffer from hallucination, where responses are generated without factual grounding. However, by retrieving relevant document chunks before generating answers, the system significantly reduces such issues and ensures higher accuracy and reliability. This makes the system particularly suitable for domains like energy, where precise and factual information is essential.The system also demonstrates strong performance in terms of efficiency and scalability. The use of vector databases enables fast similarity-based search, allowing the system to handle large volumes of data without significant delays. Additionally, the modular design of the system ensures that different components such as document processing, embedding generation, and query handling operate independently. This modularity makes the system easy to maintain, extend, and integrate with other technologies or platforms in the future.Another important contribution of this project is the reduction of manual effort and time consumption. In traditional scenarios, users are required to read through lengthy documents to find relevant information, which is both time-consuming and inefficient. The proposed system automates this process by providing direct answers to user queries, thereby improving productivity and user experience. The use of FastAPI also provides a simple and effective interface through Swagger UI, allowing users to interact with the system without the need for a dedicated frontend application.Furthermore, the project demonstrates the practical application of Generative AI in real-world scenarios. It shows how AI can be used not just for content generation, but also for intelligent information retrieval and knowledge extraction. The system can be extended to various domains beyond energy, such as healthcare, legal documents, and research analysis, making it a versatile solution.Despite its successful implementation, the system has certain limitations. It primarily supports text-based PDF documents and does not handle images or scanned documents effectively. Additionally, the system **c**urrently supports only English language queries and relies on external APIs for embedding and response generation, which may introduce dependency and cost factors. However, these limitations can be addressed in future improvements by integrating OCR technologies, multi-language support, and optimized local models.In conclusion, the project successfully demonstrates the development of an intelligent document processing system using Retrieval-Augmented Generation (RAG). It provides accurate, context-aware, and efficient answers based on uploaded documents, thereby bridging the gap between traditional search systems and modern AI-driven solutions. The system is scalable, reliable, and has strong potential for real-world applications. Overall, this project highlights the power of combining retrieval techniques with generative AI to build advanced and practical solutions for document intelligence.

CHAPTER 5:FUTURE SCOPE AND ENHANCEMENTS

The current implementation of the Energy Document Intelligence System using Retrieval-Augmented Generation (RAG) provides an efficient and intelligent solution for document-based question answering. However, with continuous advancements in Artificial Intelligence and increasing real-world requirements, there is significant scope to enhance the system further. The project can be extended to improve usability, scalability, accuracy, and security, making it more suitable for enterprise-level applications and diverse domains.One of the major areas of improvement is the development of a dedicated and advanced user interface. Currently, the system operates through FastAPI and Swagger UI, which is primarily used for testing and development. A more interactive and user-friendly interface can significantly improve usability and accessibility for non-technical users.

1. Advanced User Interface

- **A.** Develop a full frontend using React, Angular, or Streamlit
- **B.** Provide better visualization of results
- **C.** Add dashboard for document management
- **D.** Improve overall user experience

2. Multi-Language Support

- **A.** Support multiple languages for documents and queries
- **B.** Integrate translation models
- **C.** Use multilingual embeddings
- **D.** Expand global usability

3. Cloud Deployment and Scalability

- **A.** Deploy system on AWS, Azure, or GCP
- **B.** Enable access from anywhere
- **C.** Support multiple users concurrently
- **D.** Improve system scalability

4. Real-Time Data Integration

- **A.** Fetch data from APIs or databases
- **B.** Update information dynamically
- **C.** Provide real-time insights
- **D.** Improve relevance of answers

5. Enhanced Security Features

- **A.** Implement user authentication and login system
- **B.** Add role-based access control
- **C.** Secure API endpoints
- **D.** Protect sensitive data